



Створення інформаційних панелей Grafana для візуалізації показників Prometheus

Що таке Prometheus

Його спочатку розробили в SoundCloud колишні співробітники Google. Але його історія виходить за рамки цього: Google мав систему на основі динамічних контейнерів для всіх своїх систем у виробництві під назвою Borg і використовував власне рішення моніторингу під назвою Borgmon для того ж. Але в SoundCloud для цього не було альтернативи, інші рішення на той час не відповідали потребам динамічних середовищ (подібно до Kubernetes). Отже, на початку створення Prometheus почалося створення рішення з відкритим вихідним кодом на тих самих ключових принципах для аналогічного середовища. Я вважаю, що настрої автора передають це найкраще

«Вони близнюки, розділені при народженні». Kubernetes безпосередньо спирається на десятирічний досвід Google із їхньою власною системою кластерного планування Borg. Зв'язки Prometheus із Google набагато слабкіші, але він черпає багато натхнення від Borgmon, внутрішньої системи моніторингу, яку Google придумав приблизно в той самий час, що й Borg. У дуже неохайному порівнянні можна сказати, що Kubernetes — це Борґ для простих смертних, а Прометей — це Борґмон для простих смертних. Обидві є «другими системами», які намагаються повторити хороші частини, уникаючи помилок і глухих кутів своїх предків.

Що таке Prometheus

Отже, що це, по суті, Prometheus — це набір інструментів моніторингу та попередження систем із відкритим кодом (так, він дуже модульний!). Він збирає показники з налаштованих цілей через задані проміжки часу, оцінює вирази правил, відображає результати та може ініціювати сповіщення, якщо якась умова виконується.

Він був прийнятий Cloud Native Computing Foundation як другий проект, прийнятий відразу після самого Kubernetes. Сьогодні Prometheus є де-факто інструментом моніторингу, який всюди використовується разом із Kubernetes.

Бази даних часових рядів

База даних часових рядів (TSDB) — це база даних, оптимізована для даних із міткою часу або даних часових рядів. Дані часових рядів – це просто вимірювання або події, які відстежуються, контролюються, зменшуються та агрегуються з часом. Це можуть бути показники сервера, моніторинг продуктивності додатків, мережеві дані, дані датчиків, події, кліки, угоди на ринку та багато інших типів аналітичних даних. (Внутрішньо він використовує 64-бітний алгоритм стиснення XOR з плаваючою комою, який Prometheus використовує під капотом)

База даних часових рядів створена спеціально для обробки показників і подій або вимірювань із міткою часу. TSDB оптимізовано для вимірювання змін з часом. Властивості, які сильно відрізняють дані часових рядів від інших робочих навантажень даних, це керування життєвим циклом даних, узагальнення та сканування великої кількості записів.

Бази даних часових рядів

Якщо ви подивіться на найпопулярніші TSDB прямо зараз, ви побачите, що Prometheus є одним із найкращих

1. InfluxDB
2. Kdb+
3. Prometheus
4. Graphite
5. RRDTOol

6. Druid
7. OpenTSDB
8. TimescaleDB
9. FaunaDB
10. KairosDB

11. GridDB
12. Alibaba Cloud TSDB
13. DolphinDB
14. eXtremeDB
15. Amazon Timestream

Чому база даних часових рядів важлива зараз?

Бази даних часових рядів не є новими, але бази даних часових рядів першого покоління були в основному зосереджені на аналізі фінансових даних, нестабільності біржової торгівлі та систем, створених для вирішення проблем торгівлі. Але сфера її дії вже не обмежується лише цим. Фундаментальні умови обчислювальної техніки різко змінилися за останнє десятиліття. Все розділилося на частини. Монолітні мейнфрейми зникли, замінені серверами без серверів, мікросерверами та контейнерами. З такою архітектурою пошук послідовності речей, що відбуваються в реальному часі в багатьох системах, що працюють паралельно, потребує системи моніторингу, побудованої на міцній основі TSDB.

Модель Push проти Pull для збору показників

Метрики є одним із основних стандартів для будь-якої системи моніторингу, яка існує безліч різних типів. За своєю суттю метрика — це, по суті, вимірювання властивості частини програми чи системи. Метрики здійснюють спостереження, відстежуючи стан об'єкта. Ці спостереження є певним значенням або серією пов'язаних значень у поєднанні з міткою часу, яка описує спостереження, вихідні дані яких зазвичай називають даними часового ряду.

Канонічним прикладом збору показників є звернення до веб-сайту, де ми регулярно збираємо різні дані, як-от кількість відвідувань певної сторінки та місцезнаходження або джерело відвідувача певного сайту. Приклади різних типів показників:

- Лічильники
- Таймери
- Події
- Журнали

Модель Push проти Pull для збору показників

Одна метрика сама по собі часто не настільки корисна, але якщо її візуалізувати з часом, особливо в поєднанні з деякими математичними перетвореннями, метрика може дати чудове розуміння системи та її поведінки. Нижче наведено деякі загальні перетворення для показників:

- Sum
- Average
- Median
- Percentiles
- Standard deviation
- Rates of change

Модель Push проти Pull для збору показників

У моніторингу на основі Push програма, яка контролюється, стає джерелом метрик і подій, які «виштовхують» дані за певний проміжок часу або порушення обмежень. Цей підхід зазвичай надає перевагу децентралізованій структурі та має перевагу в тому, що система моніторингу не вимагає «попередньої реєстрації» контрольованого компонента, оскільки емітер надсилає дані до налаштованого пункту призначення після запуску. Але він страждає від кількох недоліків і втрачає свою чітку видимість, якщо агент на основі push вмирає або не відповідає з часом. Крім того, у цьому підході потрібно оптимізувати обсяг переданих даних (оскільки це частина програми та в певному сенсі її пропускна здатність).

Рішення на основі вилучення/опитування працюють по-іншому, оскільки вони отримують дані, які надаються через різні служби та кінцеві точки. є досить поширеними у моніторингу та історично віддають перевагу централізованій організаційній структурі, хоча це не є вимогою. У першу чергу, під час моніторингу на основі вилучення, система моніторингу запитує або запитує компонент, що контролюється, наприклад, перевіряючи хост або перевіряючи, чи працює веб-сайт.

Архітектура

Більшість компонентів Prometheus написані на Go, завдяки чому їх легко створювати та розгортати як статичні двійкові файли для всіх типів цілей/систем. Prometheus збирає показники з інструментальних завдань безпосередньо або через проміжний push-шлюз для короткострокових завдань. Він зберігає всі зібрані зразки локально та запускає правила для цих даних, щоб агрегувати та записувати нові часові ряди з наявних даних або створювати сповіщення.

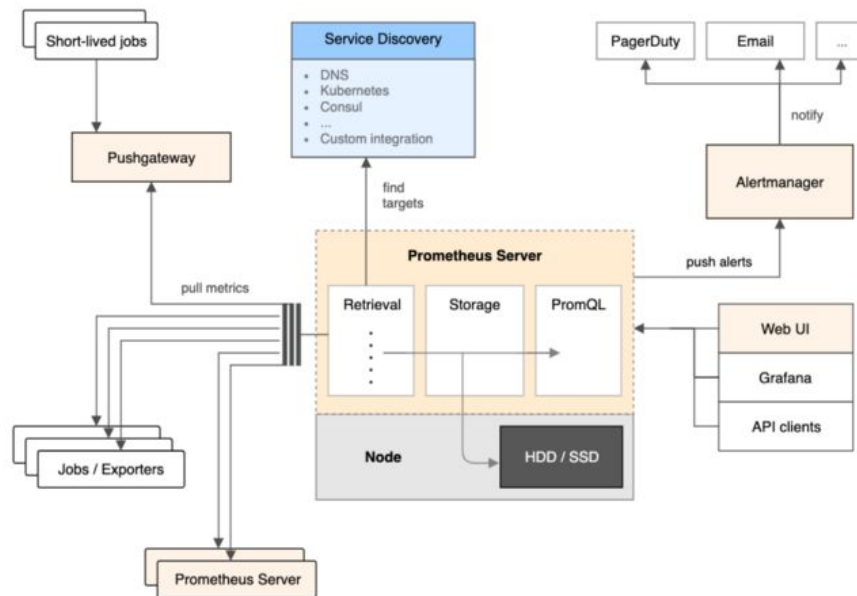
Основні риси Prometheus:

- багатовимірна модель даних із даними часових рядів, ідентифікованих назвою метрики та парами ключ/значення
- PromQL, гнучка мова запитів для використання цієї розмірності
- Збір часових рядів відбувається за допомогою моделі вилучення через HTTP
- проштовхування часових рядів підтримується через проміжний шлюз
- цілі виявляються за допомогою виявлення служби або статичної конфігурації
- підтримка кількох режимів побудови графіків і інформаційних панелей

Архітектура

Екосистема Prometheus складається з кількох компонентів, багато з яких є необов'язковими:

- головний сервер Prometheus, який збирає та зберігає дані часових рядів
- клієнтські бібліотеки для інструментування програмного коду
- push-шлюз для підтримки короткострокових завдань
- експортери спеціального призначення для таких служб, як HAProxy, StatsD, Graphite тощо, які надсилають показники з існуючої системи чи програмного забезпечення.
- менеджер сповіщень для обробки сповіщень



Коли підходить?

Prometheus добре працює для запису будь-яких чисто числових часових рядів. Він підходить як для машиноцентричного моніторингу, так і для моніторингу високодинамічних сервіс-орієнтованих архітектур, таких як Kubernetes. У світі мікросервісів його підтримка збору багатовимірних даних і запитів є особливою перевагою.

Prometheus розроблено для надійності, щоб бути системою, до якої ви звертаєтеся під час збою, щоб дозволити вам швидко діагностувати проблеми. Кожен сервер Prometheus є автономним і не залежить від мережевого сховища чи інших віддалених служб. Ви можете покластися на нього, коли інші частини вашої інфраструктури зламані, і вам не потрібно налаштовувати розгалужену інфраструктуру, щоб використовувати його.

Коли не підходить?

Prometheus цінує надійність. Ви завжди можете переглянути доступну статистику щодо вашої системи, навіть за умов збою. Якщо вам потрібна 100% точність, наприклад для виставлення рахунків за запитом, Prometheus не є гарним вибором, оскільки зібрані дані, швидше за все, не будуть достатньо детальними та повними. У такому випадку вам буде краще використовувати якусь іншу систему для збору й аналізу даних для виставлення рахунків і Prometheus для решти моніторингу.

Налаштування Prometheus

Для демонстраційних цілей ми будемо використовувати Docker, щоб зробити речі дійсно легкими та відтворюваними будь-де. Ось простий Dockerfile, який готує для нас сцену.

- Він починається з офіційного образу Ubuntu 18.04 (біонічного).
- Встановлює деякі інструменти, якими ми будемо користуватися, наприклад wget, screen і vim
- Він завантажує найновіші бінарні випуски для Prometheus, node_exporter і alertmanager (як ми вже говорили раніше, Prometheus є високомодульним)
- Він також завантажує Grafana, який пізніше буде використано для візуалізації
- Відкриває стандартні порти для відповідних служб

```
FROM ubuntu:bionic
MAINTAINER Arindam Paul, arindampaul1989@gmail.com
```

```
RUN apt-get update \
    && apt-get install -y wget \
    && apt-get install -y screen \
    && apt-get install -y vim
```

```
WORKDIR /root
```

```
RUN wget -nv https://github.com/prometheus/prometheus/releases/download/v2.18.0/prometheus-2.18.0.linux-amd64.tar.gz
```

```
RUN wget -nv https://github.com/prometheus/node_exporter/releases/download/v0.18.1/node_exporter-0.18.1.linux-amd64.tar.gz
```

```
RUN wget -nv https://github.com/prometheus/alertmanager/releases/download/v0.20.0/alertmanager-0.20.0.linux-amd64.tar.gz
```

```
RUN wget -nv https://dl.grafana.com/oss/release/grafana-6.7.3.linux-amd64.tar.gz
```

```
# node_exporter
```

```
EXPOSE 9100
```

```
# prometheus
```

```
EXPOSE 9090
```

```
# grafana
```

```
EXPOSE 3000
```

```
# alertmanager
```

```
EXPOSE 9093
```

Налаштування Prometheus

Тепер ми створимо простий файл компонування докерів, щоб підготувати наш образ докерів до запуску, єдине, що він робить, це дає нам зручне ім'я для роботи, яке відкриє декілька портів для хосту. Завдяки цьому ми можемо отримати до них доступ із хост-машини. Загалом я віддаю перевагу створенню докерів, ніж написанню довгих команд запуску докерів із параметрами.

```
version: '3'
services:
  prometheus_demo:
    build: .
    ports:
      - "9100:9100"
      - "9090:9090"
      - "3000:3000"
      - "9093:9093"
```

Коли ці два файли є, ви можете перейти до папки, що їх містить, і запустити службу з docker-compose в інтерактивному режимі. Зауважте, що --service-ports є дуже важливою опцією, вона дозволяє нам правильно прив'язувати порт (що вимкнено за замовчуванням).

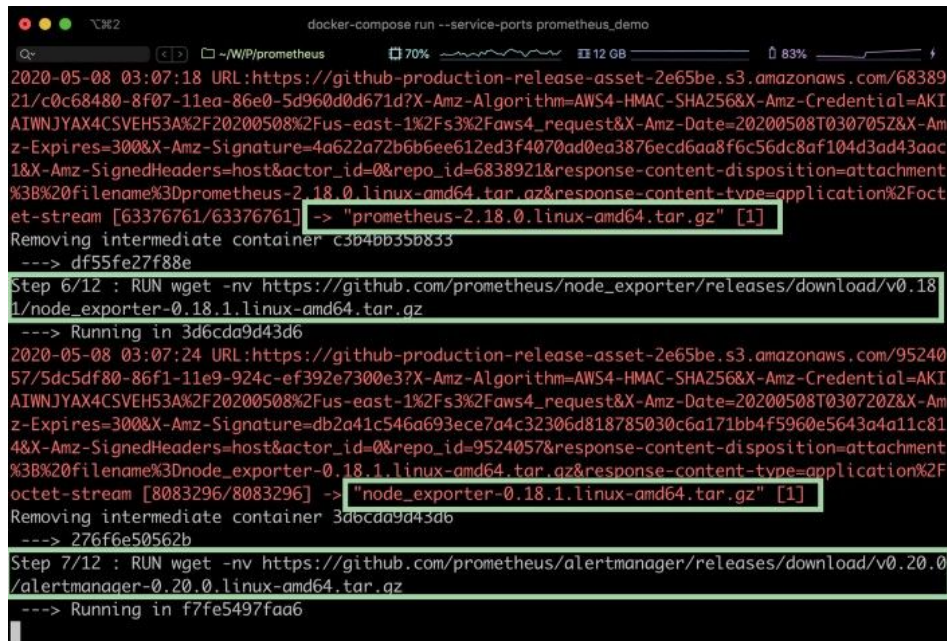
```
1 docker-compose run --service-ports prometheus_demo
```

Налаштування Prometheus

```
docker-compose run --service-ports prometheus_demo
---> c3c304cb4f22
Step 2/12 : MAINTAINER Arindam Paul, arindampaul1989@gmail.com
---> Using cache
---> 4ec8d8f212bc
Step 3/12 : RUN apt-get update && apt-get install -y wget
---> Running in fd8dcd5dc715
Get:1 http://security.ubuntu.com/ubuntu bionic-security InRelease [88.7 kB]
Get:2 http://archive.ubuntu.com/ubuntu bionic InRelease [242 kB]
Get:3 http://security.ubuntu.com/ubuntu bionic-security/main amd64 Packages [908 kB]
Get:4 http://archive.ubuntu.com/ubuntu bionic-updates InRelease [88.7 kB]
Get:5 http://archive.ubuntu.com/ubuntu bionic-backports InRelease [74.6 kB]
Get:6 http://archive.ubuntu.com/ubuntu bionic/universe amd64 Packages [11.3 MB]
Get:7 http://security.ubuntu.com/ubuntu bionic-security/multiverse amd64 Packages [8505 B]
Get:8 http://security.ubuntu.com/ubuntu bionic-security/restricted amd64 Packages [52.4 kB]
Get:9 http://security.ubuntu.com/ubuntu bionic-security/universe amd64 Packages [844 kB]
Get:10 http://archive.ubuntu.com/ubuntu bionic/main amd64 Packages [1344 kB]
Get:11 http://archive.ubuntu.com/ubuntu bionic/restricted amd64 Packages [13.5 kB]
Get:12 http://archive.ubuntu.com/ubuntu bionic/multiverse amd64 Packages [186 kB]
Get:13 http://archive.ubuntu.com/ubuntu bionic-updates/restricted amd64 Packages [66.6 kB]
Get:14 http://archive.ubuntu.com/ubuntu bionic-updates/universe amd64 Packages [1376 kB]
Get:15 http://archive.ubuntu.com/ubuntu bionic-updates/multiverse amd64 Packages [19.8 kB]
Get:16 http://archive.ubuntu.com/ubuntu bionic-updates/main amd64 Packages [1205 kB]
Get:17 http://archive.ubuntu.com/ubuntu bionic-backports/main amd64 Packages [8286 B]
Get:18 http://archive.ubuntu.com/ubuntu bionic-backports/universe amd64 Packages [8158 B]
```

Налаштування Prometheus

Щойно ви запустите команду, ви побачите купу результатів, що відповідають крокам збірки в Dockerfile. Ви також побачите, що він завантажує необхідні двійкові файли та інструменти, як показано нижче,



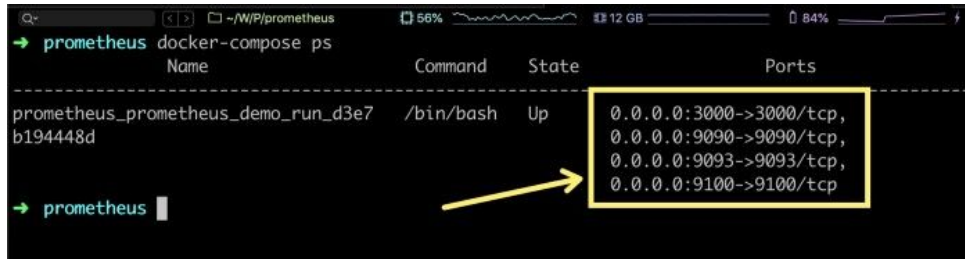
```
docker-compose run --service-ports prometheus_demo
2020-05-08 03:07:18 URL:https://github-production-release-asset-2e65be.s3.amazonaws.com/68389
21/c0c68480-8f07-11ea-86e0-5d960d0d671d?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=AKI
AIWNJYAX4CSVEH53A%2F20200508%2Fus-east-1%2Fs3%2Faws4_request&X-Amz-Date=20200508T030705Z&X-Am
z-Expires=300&X-Amz-Signature=4a622a72b6b6ee612ed3f4070ad0ea3876ecd6aa8f6c56dc8af104d3ad43aac
18X-Amz-SignedHeaders=host&actor_id=0&repo_id=6838921&response-content-disposition=attachment
%3B%20filename%3Dprometheus-2.18.0.linux-amd64.tar.gz&response-content-type=application%2Foct
et-stream [63376761/63376761] -> "prometheus-2.18.0.linux-amd64.tar.gz" [1]
Removing intermediate container c3b40b35b833
---> df55fe27f88e
Step 6/12 : RUN wget -nv https://github.com/prometheus/node_exporter/releases/download/v0.18
1/node_exporter-0.18.1.linux-amd64.tar.gz
---> Running in 3d6cda9d43d6
2020-05-08 03:07:24 URL:https://github-production-release-asset-2e65be.s3.amazonaws.com/95240
57/5dc5df80-86f1-11e9-924c-ef392e7300e3?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=AKI
AIWNJYAX4CSVEH53A%2F20200508%2Fus-east-1%2Fs3%2Faws4_request&X-Amz-Date=20200508T030720Z&X-Am
z-Expires=300&X-Amz-Signature=db2a41c546a693ece7a4c32306d818785030c6a171bb4f5960e5643a4a11c81
4&X-Amz-SignedHeaders=host&actor_id=0&repo_id=9524057&response-content-disposition=attachment
%3B%20filename%3Dnode_exporter-0.18.1.linux-amd64.tar.gz&response-content-type=application%2F
octet-stream [8083296/8083296] -> "node_exporter-0.18.1.linux-amd64.tar.gz" [1]
Removing intermediate container 3d6cda9d43d6
---> 276f6e50562b
Step 7/12 : RUN wget -nv https://github.com/prometheus/alertmanager/releases/download/v0.20.0
/alertmanager-0.20.0.linux-amd64.tar.gz
---> Running in f7fe5497faa6
```

Налаштування Prometheus

Нарешті, він перекине вас в інтерактивну підказку всередині запущеного контейнера `prometheus_demo_run`.

```
Removing intermediate container a1cd18afe94d
--> 43d8cc190894
Successfully built 43d8cc190894
Successfully tagged prometheus_prometheus_demo:latest
WARNING: Image for service prometheus_demo was built because it did not already exist. To rebuild this image you must use `docker-compose build` or `docker-compose up --build`.
root@452e3f7c04de:~#
```

Якщо ви хочете побачити зіставлення портів контейнера з хостом, ми можемо перейти до іншого терміналу, залишивши цей увімкненим (інакше запущений екземпляр завершить роботу, оскільки йому нічого робити), і запустити `docker-compose ps`, ви побачите статус свого контейнера. Наразі він знаходиться в оболонці `bash`, і стан активний, і порти відкрито, як показано нижче,

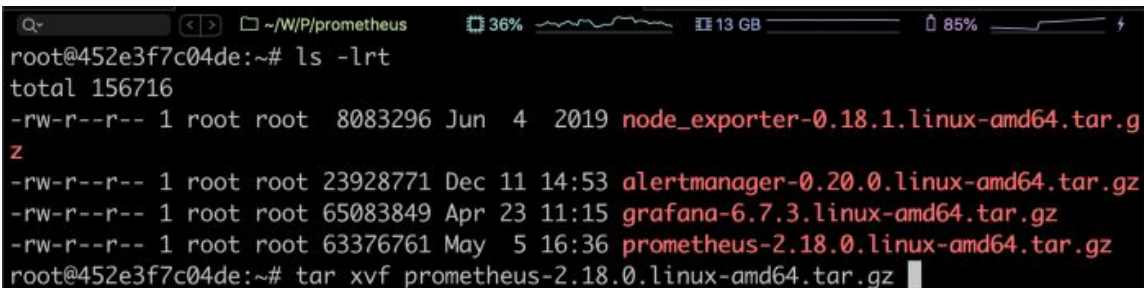


```
prometheus docker-compose ps
Name                  Command      State      Ports
-----
prometheus_prometheus_demo_run_d3e7b194448d /bin/bash    Up         0.0.0.0:3000->3000/tcp,
0.0.0.0:9090->9090/tcp,
0.0.0.0:9093->9093/tcp,
0.0.0.0:9100->9100/tcp
```

Налаштування Prometheus

Це все, що потрібно для налаштування, тепер починається веселий час. Якщо ви перерахуєте каталог, усі необхідні інструменти вже завантажено для нас.

Спочатку ми запустимо сервер Prometheus, щоб розпочати роботу, ви можете продовжити, просто роздобувши пакет.



```
root@452e3f7c04de:~# ls -lrt
total 156716
-rw-r--r-- 1 root root 8083296 Jun  4 2019 node_exporter-0.18.1.linux-amd64.tar.g
z
-rw-r--r-- 1 root root 23928771 Dec 11 14:53 alertmanager-0.20.0.linux-amd64.tar.gz
-rw-r--r-- 1 root root 65083849 Apr 23 11:15 grafana-6.7.3.linux-amd64.tar.gz
-rw-r--r-- 1 root root 63376761 May  5 16:36 prometheus-2.18.0.linux-amd64.tar.gz
root@452e3f7c04de:~# tar xvf prometheus-2.18.0.linux-amd64.tar.gz
```

Налаштування Prometheus

І виконує команду prometheus всередині. Він запускає сервер із певною конфігурацією за замовчуванням. Тут ви можете побачити, що він починається з 15 днів періоду зберігання за

```
root@452e3f7c04de:~# cd prometheus-2.18.0.linux-amd64/  
root@452e3f7c04de:~/prometheus-2.18.0.linux-amd64# ./prometheus  
level=info ts=2020-05-08T03:10:58.980Z caller=main.go:302 msg="No time or size retention was set so using the default time retention" duration=15d  
level=info ts=2020-05-08T03:10:58.980Z caller=main.go:337 msg="Starting Prometheus" version="(version=2.18.0, branch=HEAD, revision=a12e96299dcd159ea09b260f1a21e7e4b86e011d)"  
level=info ts=2020-05-08T03:10:58.980Z caller=main.go:338 build_context="(go=go1.14.2, user=root@7fbcff55abdb, date=20200505-14:26:04)"
```

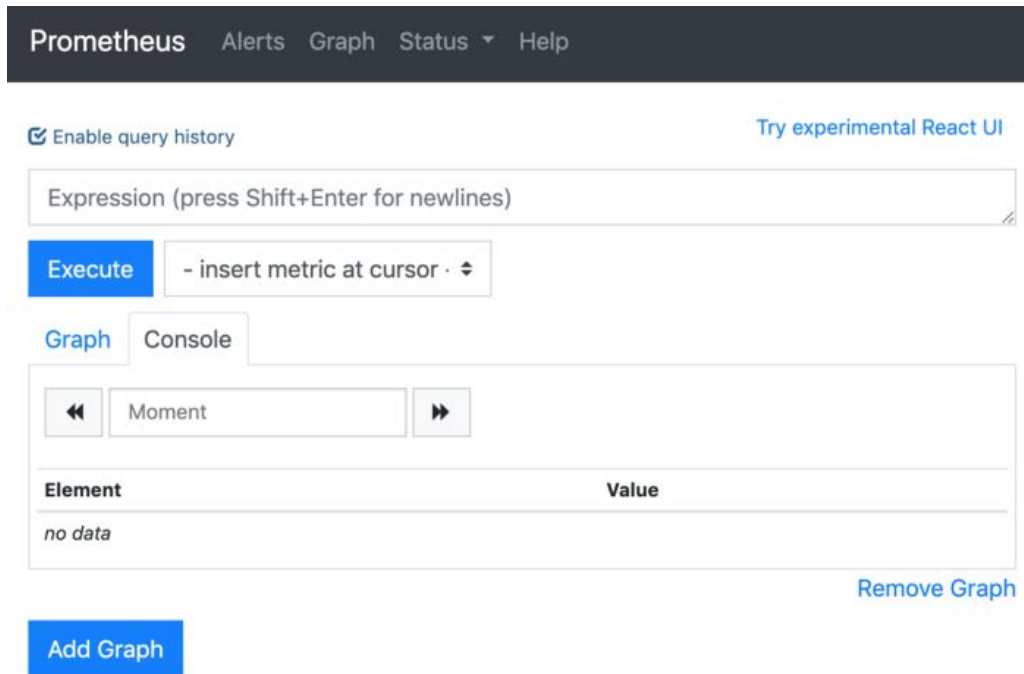
Налаштування Prometheus

Крім того, якщо ви помітили в кінці, він фактично запускає базу даних із повідомленням «TSDB розпочато», яке вказує на те, що база даних часових рядів знаходиться в основі. Нарешті, він читає файл у тому ж каталозі під назвою `prometheus.yml`, який містить конфігурацію для сервера. Ми розглянемо це більш детально в наступних розділах.

```
level=info ts=2020-05-08T03:10:59.001Z caller=main.go:694 msg="TSDB started"
level=info ts=2020-05-08T03:10:59.001Z caller=main.go:798 msg="Loading configuration file" filename=prometheus.yml
level=info ts=2020-05-08T03:10:59.003Z caller=main.go:826 msg="Completed loading of configuration file" filename=prometheus.yml
level=info ts=2020-05-08T03:10:59.003Z caller=main.go:645 msg="Server is ready to receive web requests."
```

Налаштування Prometheus

Тепер, якщо на вашому хості (ноутбуці) відкрити URL-адресу `http://localhost:9090/`, ви отримаєте щось подібне.



The screenshot displays the Prometheus web interface. At the top is a dark navigation bar with links for Prometheus, Alerts, Graph, Status, and Help. Below this, there's a toggle for 'Enable query history' and a link to 'Try experimental React UI'. The main query editor area contains a text input for the expression, an 'Execute' button, and a dropdown menu for inserting metrics. Below the editor are tabs for 'Graph' and 'Console'. The 'Graph' tab is active, showing a timeline view with a 'Moment' selector and navigation arrows. At the bottom of the graph view is a table with columns 'Element' and 'Value', currently displaying 'no data'. A 'Remove Graph' link is located at the bottom right of the graph area. A blue 'Add Graph' button is positioned at the bottom center of the interface.

Prometheus Alerts Graph Status ▾ Help

☒ Enable query history [Try experimental React UI](#)

Expression (press Shift+Enter for newlines)

Execute - insert metric at cursor · ↕

Graph Console

⏪ Moment ⏩

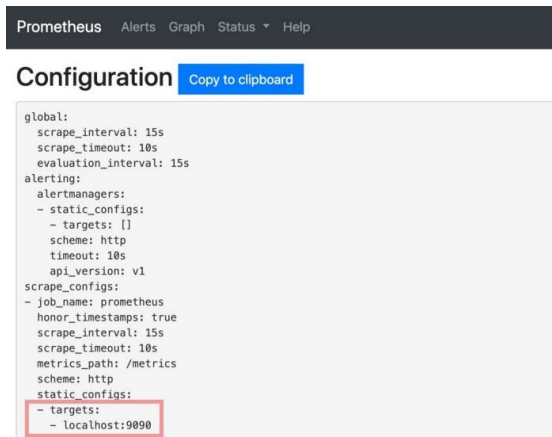
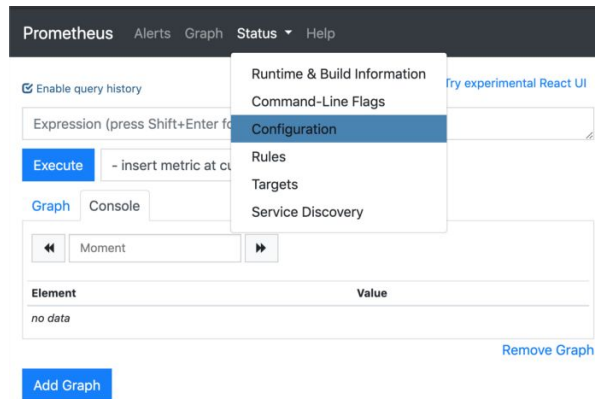
Element	Value
no data	

[Remove Graph](#)

Add Graph

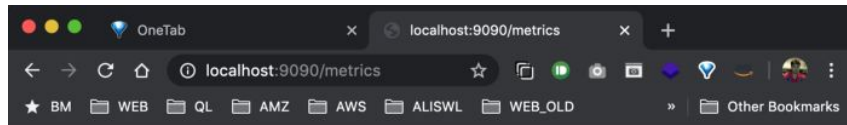
Налаштування Prometheus

Це означає, що ваш **сервер Prometheus успішно запущений** і готовий до використання. Перш ніж ми поглибимося в будь-які подробиці, я хотів би показати вам **конфігурацію**, з якою Prometheus було завантажено на першому кроці.



Налаштування Prometheus

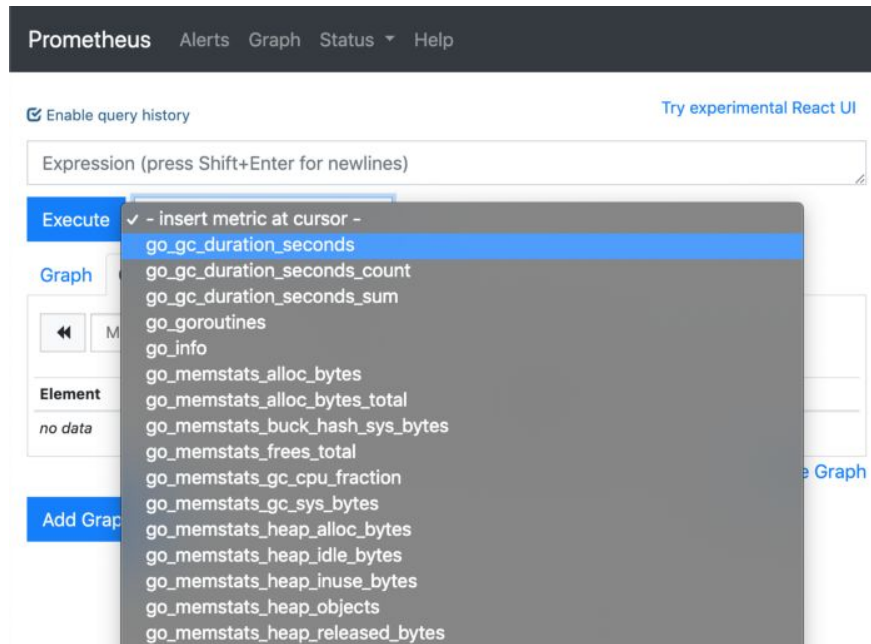
Ви помітили його дуже просту конфігурацію, як-от інтервал опитування (інтервал скачування), звідки опитувати тощо. Цікаво зауважити, що Prometheus сам надає свої метрики на тому самому порту в шляху /metrics, щоб сервер Prometheus отримував дані про власний стан. Давайте подивимося, що це таке, якщо ви відкриєте <http://localhost:9090/metrics>, безсумнівно, ви побачите прості ключі/значення з деякими коментарями.



```
# HELP go_gc_duration_seconds A summary of the pause duration of garbage collection cycles.
# TYPE go_gc_duration_seconds summary
go_gc_duration_seconds{quantile="0"} 1.8719e-05
go_gc_duration_seconds{quantile="0.25"} 9.1912e-05
go_gc_duration_seconds{quantile="0.5"} 0.000115383
go_gc_duration_seconds{quantile="0.75"} 0.000181102
go_gc_duration_seconds{quantile="1"} 0.000455282
go_gc_duration_seconds_sum 0.001319812
go_gc_duration_seconds_count 9
# HELP go_goroutines Number of goroutines that currently exist.
# TYPE go_goroutines gauge
go_goroutines 41
# HELP go_info Information about the Go environment.
# TYPE go_info gauge
go_info{version="go1.14.2"} 1
# HELP go_memstats_alloc_bytes Number of bytes allocated and still in use.
# TYPE go_memstats_alloc_bytes gauge
go_memstats_alloc_bytes 1.7997376e+07
# HELP go_memstats_alloc_bytes_total Total number of bytes allocated, even if freed.
# TYPE go_memstats_alloc_bytes_total counter
go_memstats_alloc_bytes_total 4.069408e+07
# HELP go_memstats_buck_hash_sys_bytes Number of bytes used by the profiling bucket hash table.
# TYPE go_memstats_buck_hash_sys_bytes gauge
go_memstats_buck_hash_sys_bytes 1.4642e+06
# HELP go_memstats_frees_total Total number of frees.
# TYPE go_memstats_frees_total counter
go_memstats_frees_total 170249
# HELP go_memstats_gc_cpu_fraction The fraction of this program's available CPU time used by the GC since the program started.
# TYPE go_memstats_gc_cpu_fraction gauge
go_memstats_gc_cpu_fraction 4.8087553577581346e-05
# HELP go_memstats_gc_sys_bytes Number of bytes used for garbage collection system metadata.
# TYPE go_memstats_gc_sys_bytes gauge
go_memstats_gc_sys_bytes 3.594504e+06
# HELP go_memstats_heap_alloc_bytes Number of heap bytes allocated and still in use.
```

Налаштування Prometheus

Але по суті це означає, що ми вже маємо метрики, які ми збираємо і які можна запитувати з сервера Prometheus, тож повернемося до сервера. Якщо ви клацнете спадне меню доступних показників, ви побачите всі ці показники.

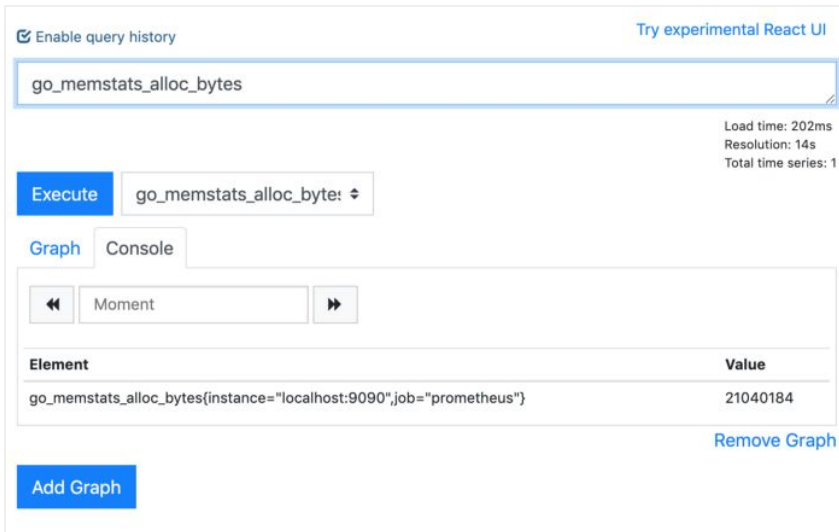


Налаштування Prometheus

Давайте виберемо метрику, щоб дослідити трохи більше, скажімо, ми хочемо знати, скільки пам'яті виділено для сервера Prometheus, ми вибираємо

`go_memstats_alloc_bytes` (Просто приклад, ви можете використовувати що завгодно).

Якщо я виконаю цей запит, ви побачите, що я отримаю одне значення. За замовчуванням будь-який запит метрики дає останній записаний результат.



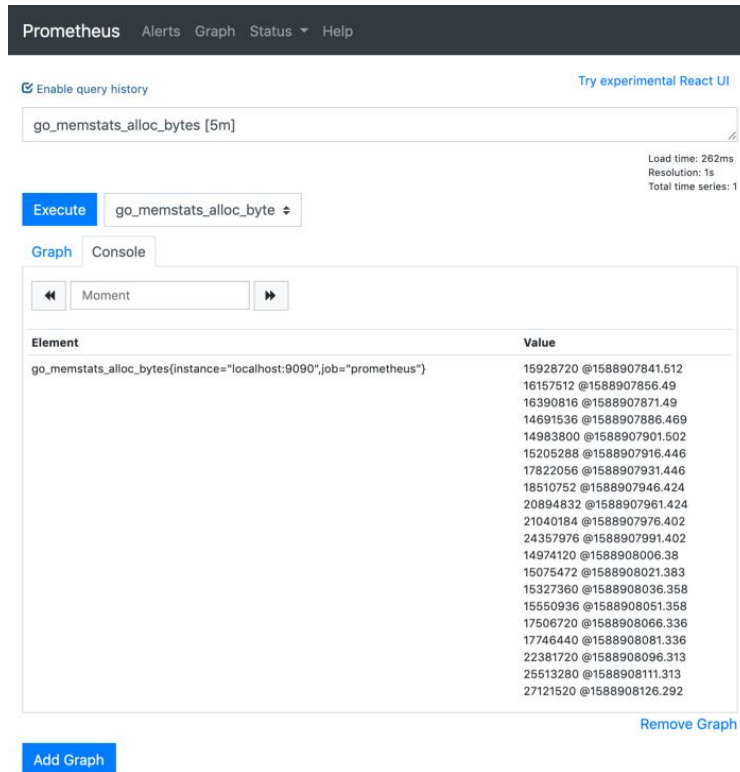
The screenshot shows the Prometheus web interface. At the top, there is a checkbox for "Enable query history" and a link for "Try experimental React UI". The query input field contains `go_memstats_alloc_bytes`. To the right of the input field, the following statistics are displayed: "Load time: 202ms", "Resolution: 14s", and "Total time series: 1". Below the input field, there is a blue "Execute" button and a dropdown menu showing `go_memstats_alloc_byte!`. Underneath, there are two tabs: "Graph" (selected) and "Console". The "Graph" tab shows a time series visualization with a "Moment" button and left/right navigation arrows. Below the graph, a table displays the query result:

Element	Value
<code>go_memstats_alloc_bytes{instance="localhost:9090",job="prometheus"}</code>	21040184

At the bottom right of the table, there is a link "Remove Graph". At the bottom left, there is a blue "Add Graph" button.

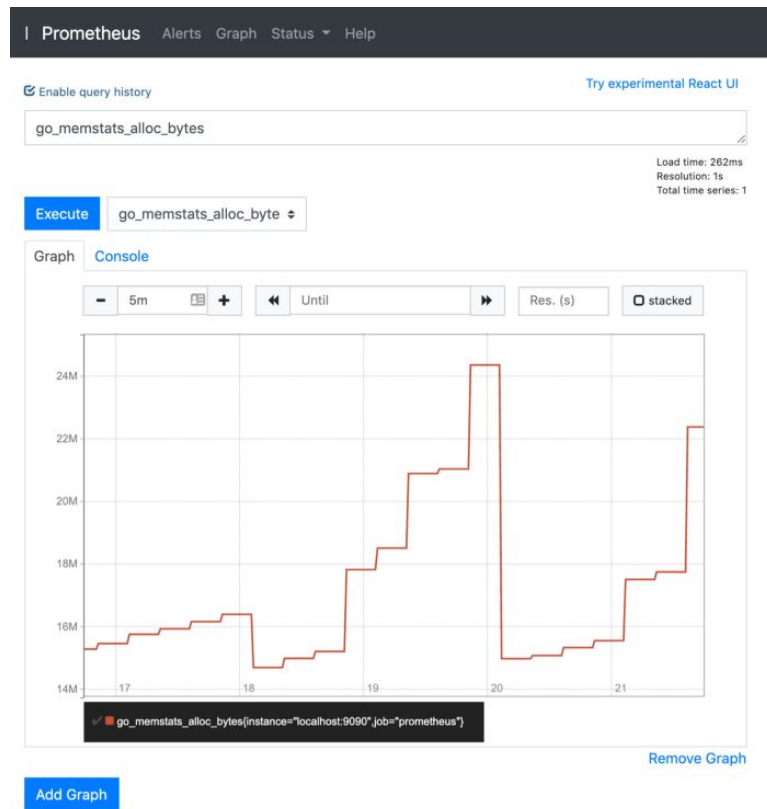
Налаштування Prometheus

Але, якщо ми хочемо знати, як це змінилося з часом, ми можемо використовувати для цього мову запитів PromQL, як показано в прикладі, її проста та виразна мова насправді така ж потужна, як сама БД часових рядів, а Prometheus — це не що інше, як ці два разом. Детальніше про це.



Налаштування Prometheus

Тепер ми отримуємо всі значення, записані за останні 5 хвилин, і ви можете бачити, що всі вони мають значення та позначку часу, пов'язану з ним. Ви також можете візуалізувати дані на вкладці візуалізації. Завдяки цьому ви можете чітко бачити, як відбувається споживання пам'яті сервером Prometheus.



Експортери в Prometheus

Існує ряд бібліотек і серверів, які допомагають експортувати існуючі метрики зі сторонніх систем як метрики Prometheus. Це корисно у випадках, коли неможливо напряму інструментувати дану систему за допомогою показників Prometheus (наприклад, MySQL Server, HAProxy або системна статистика Linux).

Prometheus має величезну екосистему експортерів, майже будь-яке популярне програмне забезпечення, яке ви хочете використовувати. Один з офіційних ми розглянемо тут докладніше

Експортер вузла (справність машини)

Node Exporter — це простий інструмент для експорту показників працездатності вашої системи. І збирає багато системних показників, дивлячись на файли /proc вашої системи. Ми вже завантажили експортер вузлів на етапі налаштування, щоб зрозуміти більше, давайте запустимо його в нашій оболонці Docker (нова оболонка використовує екран)

```
root@d925f474f53f:~# tar -xvf node_exporter-0.18.1.linux-amd64.tar.gz
node_exporter-0.18.1.linux-amd64/
node_exporter-0.18.1.linux-amd64/node_exporter
node_exporter-0.18.1.linux-amd64/NOTICE
node_exporter-0.18.1.linux-amd64/LICENSE
root@d925f474f53f:~# █
```

Експортер вузла (справність машини)

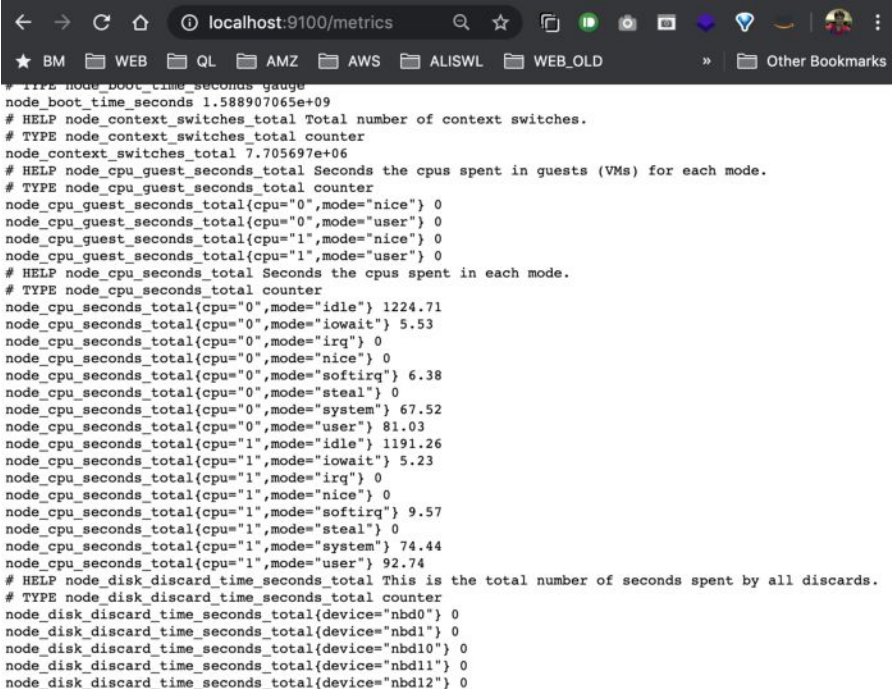
Ви можете побачити, щойно ми запускаємо експортер вузла, він запускає збирачі для багатьох багатьох окремих статистичних даних і надає їх для використання в порту 9100.

```
root@d925f474f53f:~/node_exporter-0.18.1.linux-amd64# ./node_exporter
INFO[0000] Starting node_exporter (version=0.18.1, branch=HEAD, revision=3db77732e925c08f675d7404a8c46466b2ece83e) source="node_exporter.go:156"
INFO[0000] Build context (go=go1.12.5, user=root@b50852a1acba, date=20190604-16:41:18) source="node_exporter.go:157"
INFO[0000] Enabled collectors: source="node_exporter.go:97"
INFO[0000] - arp source="node_exporter.go:104"
INFO[0000] - bcache source="node_exporter.go:104"
INFO[0000] - bonding source="node_exporter.go:104"
INFO[0000] - conntrack source="node_exporter.go:104"
INFO[0000] - cpu source="node_exporter.go:104"
INFO[0000] - cpufreq source="node_exporter.go:104"
```

```
INFO[0000] - timex source="node_exporter.go:104"
INFO[0000] - uname source="node_exporter.go:104"
INFO[0000] - vmstat source="node_exporter.go:104"
INFO[0000] - xfs source="node_exporter.go:104"
INFO[0000] - zfs source="node_exporter.go:104"
INFO[0000] Listening on :9100 source="node_exporter.go:170"
```

Експортер вузла (справність машини)

Якщо ви перейдете за URL-адресою <http://localhost:9100/metrics>, ви побачите дуже схожу сторінку метрик, яку ми бачили раніше з метриками сервера Prometheus.



```
# TYPE node_boot_time_seconds gauge
node_boot_time_seconds 1.588907065e+09
# HELP node_context_switches_total Total number of context switches.
# TYPE node_context_switches_total counter
node_context_switches_total 7.705697e+06
# HELP node_cpu_guest_seconds_total Seconds the cpus spent in guests (VMs) for each mode.
# TYPE node_cpu_guest_seconds_total counter
node_cpu_guest_seconds_total{cpu="0",mode="nice"} 0
node_cpu_guest_seconds_total{cpu="0",mode="user"} 0
node_cpu_guest_seconds_total{cpu="1",mode="nice"} 0
node_cpu_guest_seconds_total{cpu="1",mode="user"} 0
# HELP node_cpu_seconds_total Seconds the cpus spent in each mode.
# TYPE node_cpu_seconds_total counter
node_cpu_seconds_total{cpu="0",mode="idle"} 1224.71
node_cpu_seconds_total{cpu="0",mode="iowait"} 5.53
node_cpu_seconds_total{cpu="0",mode="irq"} 0
node_cpu_seconds_total{cpu="0",mode="nice"} 0
node_cpu_seconds_total{cpu="0",mode="softirq"} 6.38
node_cpu_seconds_total{cpu="0",mode="steal"} 0
node_cpu_seconds_total{cpu="0",mode="system"} 67.52
node_cpu_seconds_total{cpu="0",mode="user"} 81.03
node_cpu_seconds_total{cpu="1",mode="idle"} 1191.26
node_cpu_seconds_total{cpu="1",mode="iowait"} 5.23
node_cpu_seconds_total{cpu="1",mode="irq"} 0
node_cpu_seconds_total{cpu="1",mode="nice"} 0
node_cpu_seconds_total{cpu="1",mode="softirq"} 9.57
node_cpu_seconds_total{cpu="1",mode="steal"} 0
node_cpu_seconds_total{cpu="1",mode="system"} 74.44
node_cpu_seconds_total{cpu="1",mode="user"} 92.74
# HELP node_disk_discard_time_seconds_total This is the total number of seconds spent by all discards.
# TYPE node_disk_discard_time_seconds_total counter
node_disk_discard_time_seconds_total{device="nbd0"} 0
node_disk_discard_time_seconds_total{device="nbd1"} 0
node_disk_discard_time_seconds_total{device="nbd10"} 0
node_disk_discard_time_seconds_total{device="nbd11"} 0
node_disk_discard_time_seconds_total{device="nbd12"} 0
```

Експортер вузла (справність машини)

Цього разу, як ви бачите, чи ключі починаються з node_, що розрізняє простір ключів.

Тепер наступним кроком є те, як збирати ці показники на сервері Prometheus. Для цього ми зупинимо наш сервер Prometheus (я використовую екран, щоб відкрити кілька оболонок під час сеансу) і відредагуємо наш конфігураційний файл таким чином,

```
# metrics_path defaults to '/metrics'
# scheme defaults to 'http'.

static_configs:
- targets: ['localhost:9090', 'localhost:9100']

~
"prometheus.yml" 29L, 943C                                27,0-1          Bot
```

Експортер вузла (справність машини)

Ми додали ще одну ціль для нашого сервера, тепер сервер опитуватиме обидві ці цілі у вказаний проміжок часу для збору показників.

Таким чином ми запускаємо сервер Prometheus і перевіряємо, чи конфігурація завантажена успішно

Configuration

[Copy to clipboard](#)

```
global:
  scrape_interval: 15s
  scrape_timeout: 10s
  evaluation_interval: 15s
alerting:
  alertmanagers:
    - static_configs:
      - targets: []
      scheme: http
      timeout: 10s
      api_version: v1
scrape_configs:
  - job_name: prometheus
    honor_timestamps: true
    scrape_interval: 15s
    scrape_timeout: 10s
    metrics_path: /metrics
    scheme: http
    static_configs:
      - targets:
        - localhost:9090
        - localhost:9100
```

Metrics Path

Target Locations where metrics are available

Ви можете бачити, що зараз у розділі цілей ми маємо дві цілі, як і очікувалося. У наступних розділах ми докладніше розглянемо запити та візуалізацію лише на основі даних експортера цього вузла.

Запити Prometheus

Вся причина, чому ми вибрали БД часових рядів, як Prometheus, полягає в тому, що ми можемо надсилати запити до даних у дуже гнучкий спосіб. І Prometheus виконує цю обіцянку завдяки своїй потужній мові PromQL. Давайте спочатку розглянемо просту метрику

The screenshot shows the Prometheus web interface. At the top is a navigation bar with 'Prometheus', 'Alerts', 'Graph', 'Status', and 'Help'. Below this is a toolbar with 'Enable query history' and a link to 'Try experimental React UI'. The main query input field contains 'node_cpu_seconds_total'. To the right of the input, it shows 'Load time: 282ms', 'Resolution: 14s', and 'Total time series: 16'. Below the input is a blue 'Execute' button and a dropdown menu showing 'node_cpu_seconds_total'. Underneath are tabs for 'Graph' and 'Console'. The 'Graph' tab is active, showing a timeline with 'Moment' and navigation arrows. Below the graph is a table with two columns: 'Element' and 'Value'.

Element	Value
node_cpu_seconds_total{cpu="0",instance="localhost:9100",job="prometheus",mode="idle"}	1538.1
node_cpu_seconds_total{cpu="0",instance="localhost:9100",job="prometheus",mode="iowait"}	6.18
node_cpu_seconds_total{cpu="0",instance="localhost:9100",job="prometheus",mode="irq"}	0
node_cpu_seconds_total{cpu="0",instance="localhost:9100",job="prometheus",mode="nice"}	0
node_cpu_seconds_total{cpu="0",instance="localhost:9100",job="prometheus",mode="softirq"}	7.16
node_cpu_seconds_total{cpu="0",instance="localhost:9100",job="prometheus",mode="steal"}	0
node_cpu_seconds_total{cpu="0",instance="localhost:9100",job="prometheus",mode="system"}	78.23
node_cpu_seconds_total{cpu="0",instance="localhost:9100",job="prometheus",mode="user"}	92.08
node_cpu_seconds_total{cpu="1",instance="localhost:9100",job="prometheus",mode="idle"}	1480.1
node_cpu_seconds_total{cpu="1",instance="localhost:9100",job="prometheus",mode="iowait"}	5.83

Запити Prometheus

Ви можете побачити за допомогою експортера вузлів, він збирає вкладену інформацію про те, який хост і примірник надсилає ці дані.

Важливо зауважити, що якщо ви додаєте більше вузлів і експортуєте дані за допомогою засобу експорту вузлів, показники матимуть однакові ключі, але відрізнятимуться у вкладеному контексті, як зазначено вище, де екземпляр або процесор відрізнятимуться. Отже, у Prometheus показники збирає служба, а не мій хост. Дуже важливо мати це на увазі.

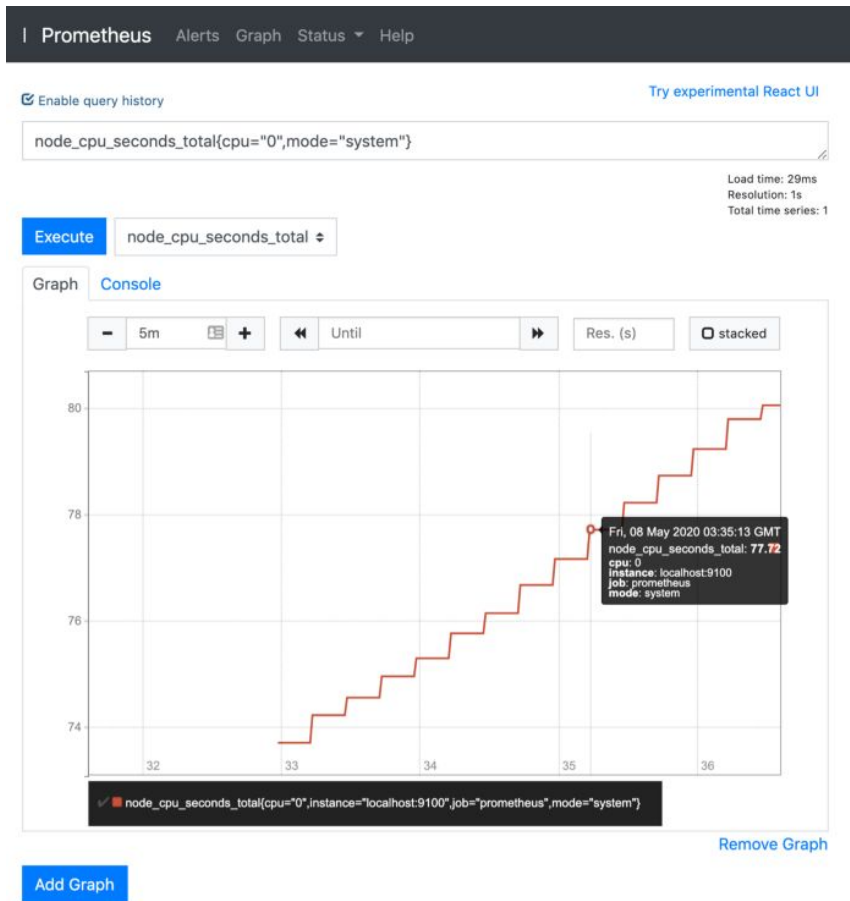
Ми можемо додати більше умов до запиту для деталізації певного показника, як цей, це дає нам останнє зареєстроване значення для показника

The screenshot shows the Prometheus web interface. At the top is a navigation bar with 'Prometheus', 'Alerts', 'Graph', 'Status', and 'Help'. Below this is a toggle for 'Enable query history' and a link to 'Try experimental React UI'. The main query input field contains the query: `node_cpu_seconds_total{cpu="0",mode="system"}`. To the right of the input field, it displays: 'Load time: 214ms', 'Resolution: 14s', and 'Total time series: 1'. Below the input field is an 'Execute' button and a dropdown menu showing 'node_cpu_seconds_total'. There are two tabs: 'Graph' (selected) and 'Console'. The 'Graph' tab shows a time range selector set to 'Moment'. Below the graph area is a table with two columns: 'Element' and 'Value'. The table contains one row with the query and the value '79.24'. At the bottom right of the table is a 'Remove Graph' link. At the bottom left is an 'Add Graph' button.

Element	Value
<code>node_cpu_seconds_total{cpu="0",instance="localhost:9100",job="prometheus",mode="system"}</code>	79.24

Запити Prometheus

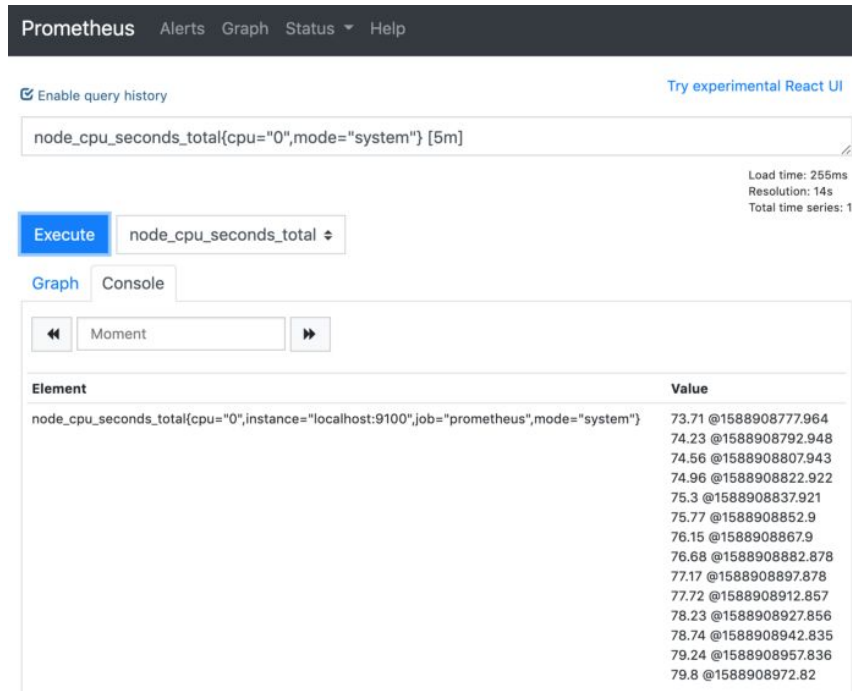
Ми також можемо візуалізувати ті самі дані на вкладці «Графік», яка показує дані, зібрані за певний час



Запити Prometheus

Якщо ви хочете побачити фактичні точки даних, ми можемо додати часовий проміжок [5 м] до нашого запиту та виконати те саме. Ось значення для останнього 5-х хвилинного інтервалу, показані на діаграмі нижче.

Отже, за допомогою цього простого налаштування ви можете побачити, скільки ви можете отримати безкоштовно, не виконуючи майже жодної роботи. Prometheus також підтримує публікацію спеціальних показників у дуже потужний спосіб усіма можливими мовами. Величезна спільнота знову процвітає завдяки аспектам вибору та гнучкості для комбінування та поєднання ваших рішень.



Grafana

Тепер, коли у нас є сервер Prometheus, який збирає для нас метрики, нам потрібен дуже потужний інструмент візуалізації, який можна використовувати для всіх видів інформаційних панелей і моніторингу.

Grafana є чудовим інструментом у цьому відношенні. Він інтегрується майже з усіма популярними джерелами серверів показників, як-от Prometheus, і дає нам набагато більше візуальних інструментів, які зрештою перетворюють їх на складні запити до сервера Prometheus, але привіт! нам не потрібно турбуватися про них.

Отже, давайте витягнемо завантажений Grafana в нову оболонку (використовуйте Screen для створення

```
root@d925f474f53f:~/grafana-6.7.3# ls
LICENSE NOTICE.md README.md VERSION bin conf public scripts tools
root@d925f474f53f:~/grafana-6.7.3# cd bin/
root@d925f474f53f:~/grafana-6.7.3/bin# ls
grafana-cli grafana-cli.md5 grafana-server grafana-server.md5
root@d925f474f53f:~/grafana-6.7.3/bin# ./grafana-server
```

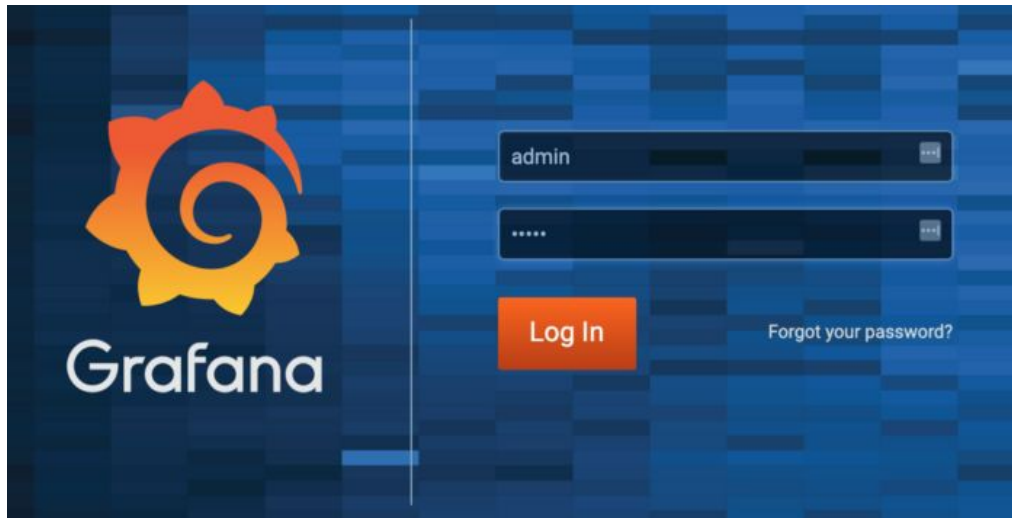
Grafana

Він запустить сервер Grafana на стандартному порту 3000.

```
INFO[05-08|03:40:01] Initializing UsageStatsService      logger=server
INFO[05-08|03:40:01] Initializing CleanUpService         logger=server
INFO[05-08|03:40:01] Initializing NotificationService     logger=server
INFO[05-08|03:40:01] Initializing provisioningServiceImpl logger=server
INFO[05-08|03:40:01] Initializing Stream Manager
INFO[05-08|03:40:01] HTTP Server Listen                  logger=http.server address=[::]:3000 protocol=http subUrl= socket=
INFO[05-08|03:40:01] Backend rendering via phantomJS      logger=rendering renderer=phantomJS
WARN[05-08|03:40:01] phantomJS is deprecated and will be removed in a future release. You should consider migrating from phantomJS to grafana-image-renderer plugin. Read more at https://grafana.com/docs/grafana/latest/administration/image_rendering/ logger=rendering renderer=phantomJS
```

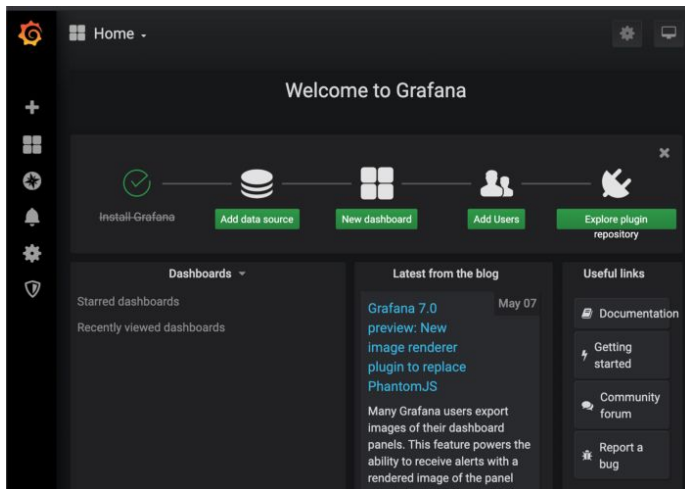
Grafana

Якщо ми перейдемо на `http://localhost:3000`, то потрапимо на домашню сторінку grafana. Ми можемо увійти в grafana за допомогою імені користувача за замовчуванням `admin` і пароля `admin`. (Для користувачів, які вперше залучаються, він може спочатку попросити вас змінити пароль, не соромтеся встановити його так, як вам подобається)

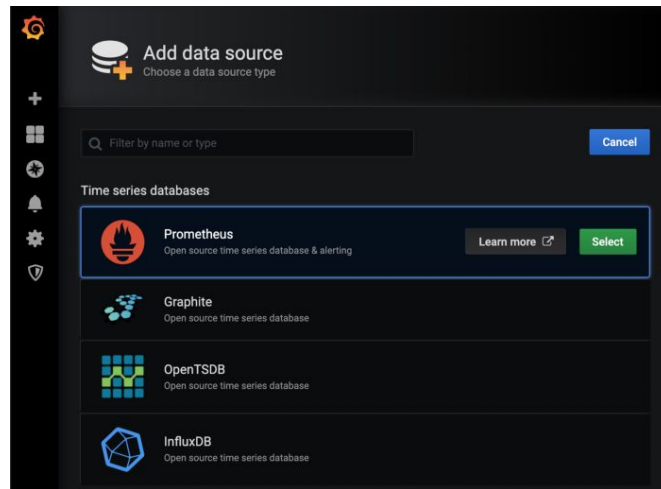


Grafana

Увійшовши в систему, ви побачите, що Grafana перше потребує джерела даних для роботи. У нашому випадку ми продовжимо й натиснемо посилання додати джерело даних.

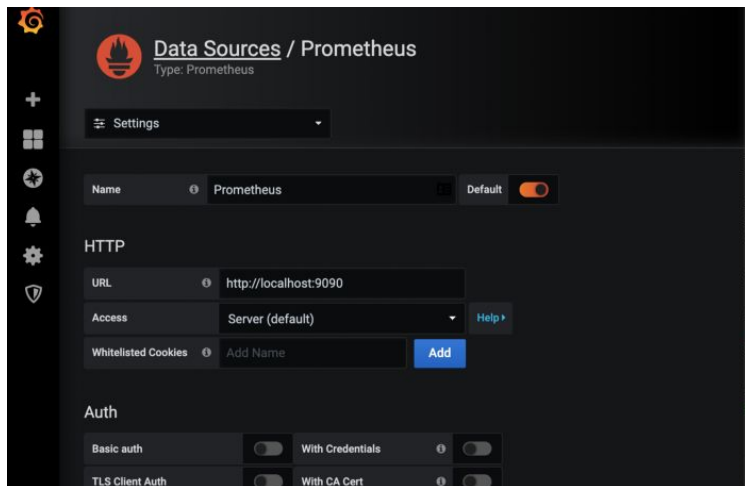


У параметрах ви можете побачити, що Grafana здатний спілкуватися з кількома базами даних, Prometheus знаходиться на вершині :)

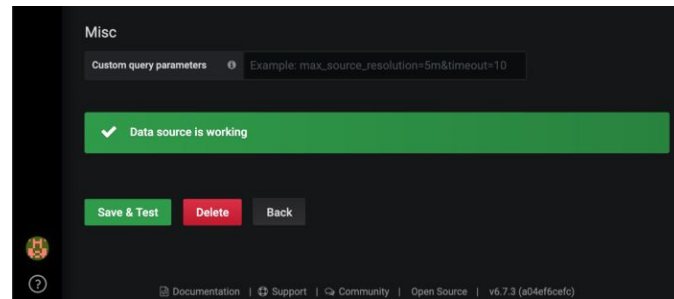


Grafana

Тому ми вибираємо Prometheus і надаємо URL-адресу нашого сервера Prometheus.

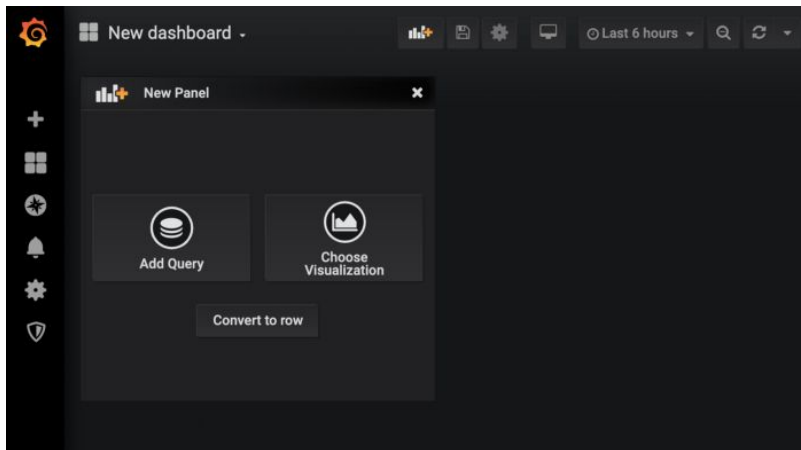


Ми можемо зберегти та перевірити конфігурацію, щоб переконатися, що джерело даних працює належним чином.

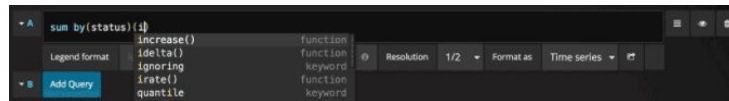
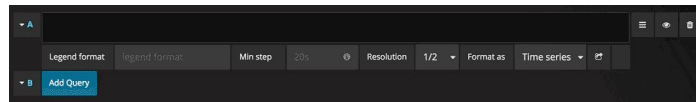


Grafana

Після завершення цього налаштування ми тепер можемо використовувати Grafana для створення інформаційних панелей і нагадувань (так, нагадування можна створювати в самому шарі Grafana, хоча ми покажемо вам менеджер сповіщень, який працює разом із Prometheus)

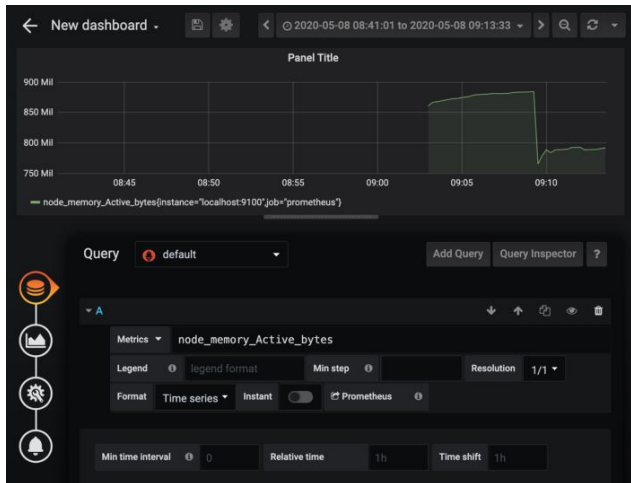


Для розваги, давайте додамо Запит для використання пам'яті всього вузла. У Grafana є дуже потужна функція автозаповнення, просто для ілюстрації, ось як ви отримувате автозаповнення в інтерфейсі користувача,



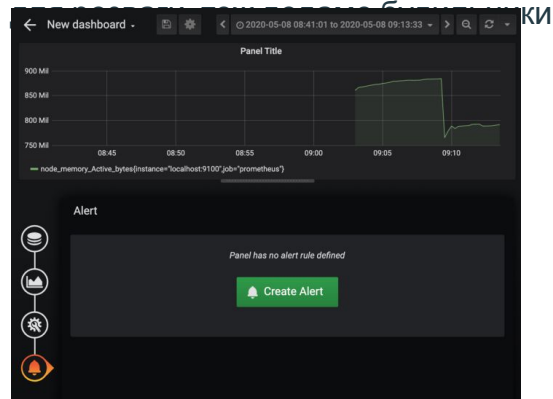
Grafana

Отже, якщо ми почнемо вводити `node_`, ми отримаємо купу автозавершень, поки що ми будемо використовувати метрику `node_memory_Active_bytes`, щоб побачити використання пам'яті.



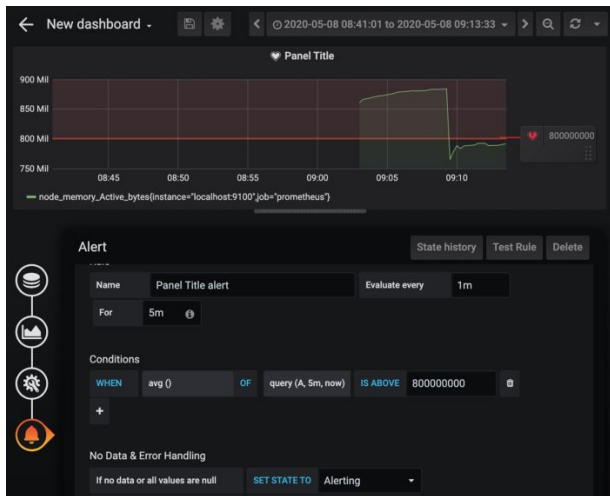
Відразу ви можете побачити, як дані починають надходити у гарний графік. Поаплодуйте собі, якщо все вийшло.

Але це ще не кінець, як ми вже говорили, за допомогою Grafana ми можемо додавати сигнали тривоги прямо там, де налаштовано графіки. Давайте зробимо один



Grafana

І налаштуйте умови сигналізації, порогові значення, ви можете побачити, як індикатор тривоги оживає, показуючи вам, де саме встановлюється будильник.

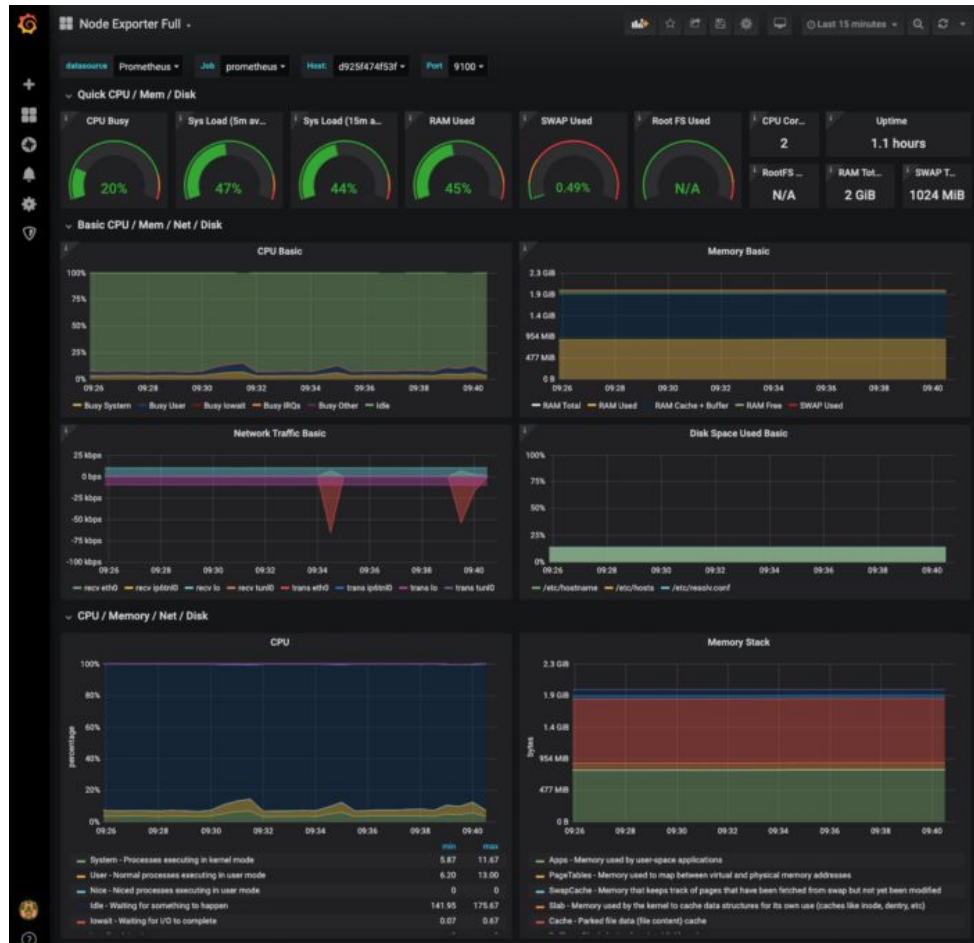


Те, що ми дізналися, є лише верхівкою айсберга щодо того, що ви можете зробити з Grafana. Наприклад, панель інструментів Grafana можна експортувати та імпортувати у форматі JSON (повністю доступний). І знову ж таки, є величезна спільнота, яка вже поділилася багатьма інформаційними панелями, які ми можемо повторно використовувати для вивчення та налаштування відповідно до наших потреб.

У цьому випадку для цього простого налаштування, якщо ми хочемо побачити повну візуалізацію на комплексній інформаційній панелі, я використав повну інформаційну панель спільноти Node Exporter.

І альт!! Це те, що ми отримуємо від нього за замовчуванням, чи це не диво :)

Grafana



Висновок

Отже, ми дізналися кілька речей, зокрема, що таке Prometheus і як він працює, як його налаштувати, як ми можемо отримувати показники від експортерів і запитувати їх. Як ми можемо візуалізувати те саме в Grafana.

Питання та відповіді